

OR114-2 Midterm Project

Problem 1

1. Mathematical Formulation

Sets

- C : Set of vehicles.
- O : Set of customer orders.
- p_i : Pickup station of order i .
- r_i : Return station of order i .
- s_i : Pick-up time of order i .
- e_i : Return time of order i .
- l_i : Requested vehicle level for order i .
- R_i : Revenue generated by order i .
- h_c : Initial station of vehicle c .
- q_c : Vehicle level of vehicle c .

Parameters

- A^0 : set of feasible initial assignments
- A : set of feasible order transitions
- m_{ci} : relocation time from vehicle c to order i
- m_{ij} : relocation time between orders i and j
- B : relocation time budget

The feasible sets A^0 and A are defined by IEDO timing constraints, where a transition (i, j) is valid only if:

$$RE_i + 240 + T_{ab} \leq PU_j - 30$$

where RE_i , T_{ab} , and PU_j represent return time, travel time, and pickup time, respectively.

Decision Variables

$$y_i = \begin{cases} 1, & \text{if order } i \text{ is accepted} \\ 0, & \text{otherwise} \end{cases}$$
$$x_{ci} = \begin{cases} 1, & \text{if vehicle } c \text{ starts with order } i \\ 0, & \text{otherwise} \end{cases}$$
$$z_{cij} = \begin{cases} 1, & \text{if vehicle } c \text{ serves order } j \text{ after } i \\ 0, & \text{otherwise} \end{cases}$$

Objective Function

$$\max \sum_{i \in O} R_i y_i$$

Constraints

(1) Order assignment constraint

$$\sum_{(c,i) \in A^0} x_{ci} + \sum_{(c,j,i) \in A} z_{cji} = y_i \quad \forall i \in O$$

(2) Vehicle starting constraint

$$\sum_{i:(c,i) \in A^0} x_{ci} \leq 1 \quad \forall c \in C$$

(3) Flow conservation constraint

$$\sum_{j:(c,i,j) \in A} z_{cij} \leq x_{ci} + \sum_{k:(c,k,i) \in A} z_{cki} \quad \forall c \in C, i \in O$$

(4) Relocation budget constraint

$$\sum_{(c,i) \in A^0} m_{ci} x_{ci} + \sum_{(c,i,j) \in A} m_{ij} z_{cij} \leq B$$

2. Optimal Solution for Instance 05

The optimization model accepts 8 out of 10 orders, including Orders 3, 4, 5, 6, 7, 8, 9, and 10. Orders 1 and 2 are rejected. The accepted sales revenue is 117,000, and the final profit after rejection compensation costs is 106,800. The total relocation time used is 1,080 minutes, which satisfies the relocation budget limit ($1080 \leq 1200$). Only one vehicle upgrade is required in the final solution, where a level-2 vehicle is assigned to Order 6, which requests a level-1 vehicle.

3. Operational Plan

Order	Car	Upgrade	Pickup → Return	Rental Period	Revenue
3	3	No	5 → 1	01/02 03:30 – 01/04 11:30	5,600
4	1	No	1 → 2	01/03 12:00 – 01/05 15:00	5,100
5	6	No	6 → 6	01/01 00:00 – 01/04 20:00	36,800
6	4	Yes	3 → 4	01/04 23:00 – 01/05 14:00	1,500
7	5	No	2 → 1	01/02 03:30 – 01/04 23:30	27,200
8	6	No	6 → 6	01/05 04:30 – 01/05 14:30	4,000
9	4	No	3 → 4	01/01 19:30 – 01/04 15:30	27,200
10	2	No	4 → 2	01/01 06:00 – 01/05 06:00	9,600

4. Vehicle Relocation Plan

Car	From	To	Depart	Arrive	Minutes
2	2	4	01/01 00:00	01/01 04:00	240
3	3	5	01/01 00:00	01/01 03:30	210
4	5	3	01/01 00:00	01/01 03:30	210
4	4	3	01/04 19:30	01/04 22:30	180
5	4	2	01/01 00:00	01/01 04:00	240

Problem 2

The required deliverable for this problem is the Python program. Our submitted function reads one instance file and returns the two required objects: **assignment**, a one-dimensional list indicating which car serves each order, and **relocation**, a two-dimensional list describing employee car movements. The implementation is written for the required Python interface and is designed to return a feasible plan within the three-minute limit. All design rationale, pseudocode, and complexity analysis are reported in Problem 3.

Problem 3

The hidden instances are graded under a three-minute time limit. This constraint is the main reason for our design. A full IP can make globally good decisions, but it is too expensive on large instances. A purely greedy method is fast, but it can waste scarce relocation minutes early. We therefore divide the algorithm into two parts:

- **Pre-build:** construct strong feasible solutions very quickly using two different approaches proposed by our team.
- **Optimization:** spend the remaining time on batch local IP repair, improving only small neighborhoods of the current solution.

In implementation, the pre-build stage is intended to finish quickly, and every local IP call in the optimization stage receives only a short time limit. The outer loop always checks the wall-clock deadline before starting another repair.

Pre-build Approach A: Revenue-first route insertion

Approach A is designed to be a fast and reliable high-profit route generator. It uses a single-pass greedy insertion process:

- **Order prioritization.** Orders are sorted primarily by decreasing revenue and secondarily by pickup time. The purpose is to give high-value orders early access to cars and relocation budget.
- **Chronological insertion search.** Each car maintains a chronological route. For an order k , the method scans compatible cars, i.e., exact level matches and valid one-level upgrades, and uses the pickup time to locate where k would be inserted in the route.
- **Feasibility evaluation.** The transition from the predecessor order to k and the transition from k to the successor order are checked. These tests enforce the one-hour possible late return, three-hour cleaning, thirty-minute readiness requirement, station moving time, level compatibility, and remaining global relocation budget.
- **Lexicographical selection.** Among all feasible insertions for the current order, the selected insertion minimizes marginal moving time first, upgrade penalty second, local idle time third, and car ID last.

If i and j are the predecessor and successor around k , the marginal moving time is

$$\Delta_{\text{move}} = T_{r_i, p_k} + T_{r_k, p_j} - T_{r_i, p_j},$$

with the obvious modification when k is inserted as the first or last order of a car route. The exact key used by the selection rule is

$$(\Delta_{\text{move}}, \text{upgrade penalty}, \text{local idle time}, \text{car ID}).$$

Thus Approach A prioritizes revenue while still conserving relocation budget and avoiding unnecessary upgrades. For very small instances, an exact solver may be used as a fallback, but this step is bypassed for large hidden instances. Let $\bar{r}$ be the average route length. Sorting orders costs $O(n_K \log n_K)$. For each order, the algorithm may check n_C cars and inspect a local insertion position, so the overall complexity is approximately $O(n_K n_C \bar{r})$ in the worst case.

Pre-build Approach B: Demand-aware look-ahead

Approach B is intended to fix the main weakness of simple greedy assignment: it looks beyond the immediate order. The planning horizon is divided into six-hour buckets. For each station, car level, and bucket, we estimate future demand value. A car of level l contributes to the ability to serve level l and level $l-1$ orders, reflecting the free one-level upgrade rule. The table is computed by adding the value of each future pickup to its pickup station and requested level, then using a rolling four-bucket window, i.e., about 24 hours, to estimate how useful a car will be near a station in the near future.

When evaluating whether car c should serve order k , Approach B uses a score of the following form:

$$3R_k + w_f F(\text{return station}, q_{\text{after}}) - w_o F(\text{current station}, q_{\text{now}}) - w_m M(1 + 3B_{\text{used}}/B) - w_i I - w_u U.$$

Here M is the relocation time, I is idle time, and U is the upgrade penalty. The term $F(\cdot)$ is the future demand score. The positive future term rewards returning a car to a useful future location, while the opportunity cost penalizes removing a car from a station that will soon need it. The factor $3R_k$ appears because total profit can be rewritten as

$$\sum_{k \in A} R_k - 2 \sum_{k \notin A} R_k = 3 \sum_{k \in A} R_k - 2 \sum_k R_k,$$

so accepting one more order improves the objective by $3R_k$ relative to rejecting it.

Approach B is then executed through a sequence of phases:

- **Initial greedy assignment.** Several parameter variants are tried, such as bucketed density order, chronological order, revenue-density order, and revenue-first order. Each variant greedily assigns orders to cars using the score above.
- **Shortage-bucket repair.** Rejected high-value orders are grouped by pickup station, time bucket, and requested level. Buckets with high remaining value and low expected inbound relocation effort are repaired first.
- **Limited route insertion.** High-value rejected orders are tested for direct insertion into existing car routes. Only nearby chronological positions and a capped number of candidate cars are checked.
- **One-removal swaps.** If direct insertion is impossible, the method tries replacing one lower-value accepted order in a compatible route with a higher-value rejected order.

- **Limited local replacement.** As a final lightweight repair, the method tries replacing the last order on a compatible car route, which is a cheap way to correct some greedy end-of-route mistakes.

Each proposed repaired route is simulated from the car’s initial station and ready time, so timing, level compatibility, and relocation budget are checked again before the route replaces the old one. On very small instances, a bounded exact DFS is used as a final improvement; this is automatically skipped on large hidden instances.

If Q is the number of six-hour buckets, building the look-ahead table costs $O(n_K + n_S n_L Q)$. The greedy assignment scan is $O(n_K n_C)$ for each parameter variant. Repair loops are capped by constants in the implementation, so their cost is bounded for grading-time control. The pre-build stage runs both Approach A and Approach B and keeps the feasible solution with larger profit.

Optimization: batch local IP repair

The pre-build stage is fast, so we use the remaining time to improve the solution by exact optimization on small neighborhoods. First, the assignment is converted into routes for all cars. In each iteration, we sample a small batch of cars whose routes are relatively inefficient. The orders currently served by these cars are released, while routes of all other cars remain fixed. We then add a capped list of compatible high-revenue candidate orders, with priority given first to the released orders and then to currently rejected orders with large revenue. This keeps the local IP small while still giving it a chance to recover from a bad earlier assignment.

For this batch, we solve a local network-flow IP. It uses start variables for a selected car’s first order, link variables for consecutive orders on a selected car, and acceptance variables for candidate orders. The local constraints are the same logical constraints as in Problem 1: each accepted order has one incoming arc, each selected car starts at most one route, flow is conserved, and the moving time used by the repaired batch does not exceed the remaining budget. If the local IP returns a set of routes with higher total profit and feasible moving time, we replace the old routes; otherwise, the incumbent solution is kept. Since the incumbent is never overwritten by a worse or infeasible repair, the algorithm can stop at any time and still return a feasible plan.

The full procedure is:

```

Input: instance file and three-minute time limit.
1.    $A^{(1)} \leftarrow$  Approach A construction.
2.    $A^{(2)} \leftarrow$  Approach B construction.
3.    $A \leftarrow \arg \max\{\text{profit}(A^{(1)}), \text{profit}(A^{(2)})\}$ .
4.   while time remains do
5.     sample a batch of low-efficiency car routes;
6.     release those routes and build a candidate order set;
7.     solve the local IP with a short per-IP time limit;
8.     accept the repair only if profit improves;
9.   return  $A$  and its relocation plan.

```

Suppose a local batch has b cars and at most m candidate orders. The local IP contains $O(bm)$ start arcs and $O(bm^2)$ order-to-order arcs in the worst case, so it has $O(bm^2)$ binary variables and constraints. In our implementation both b and m are capped, and each local IP has a short time limit τ . Therefore, under total time budget T , the number of expensive repair attempts is at most on the order of T/τ , plus the cheaper candidate selection and route-conversion overhead. Thus the algorithm uses exact optimization where it is most useful, while the total running time is controlled by the three-minute wall-clock limit.

Problem 4

To make our algorithm convincing beyond the five public instances, we generated random instances with our scenario generator. The planning horizon always starts at 2023/01/01 00:00. The fixed values are $n_S = 100$ stations and $n_L = 10$ levels. For every instance, n_D is sampled uniformly from 7 to 100. Hourly rates are generated as a strictly increasing sequence: the first base rate starts from a random value between 150 and 250, and each next level adds a random increment between 50 and 150. Pickup times are rounded to 30-minute slots and generated within the first $n_D - 2$ days so that rentals can finish inside the horizon; rental duration is uniformly sampled from 2 to 48 hours. Moving time is symmetric with $T_{ii} = 0$ and $T_{ij} = 90$ minutes for $i \neq j$.

The generator creates fourteen scenarios. Unless stated otherwise, $n_C = 100$. In the high-load scenario S4, $n_C = 80$ and $n_K = 3n_C$; in the other scenarios, n_K equals the listed load ratio times n_C . Level distribution is uniform except S5, where orders are mostly low odd levels while cars are mostly high even levels. Station distribution is uniform except S6, where orders move from even pickup stations to odd return stations, and S7–S8, where returns are concentrated around one or two hub groups. Time distribution is uniform except S9, which weights weekend days, and S10–S11, which create one or three normal peak centers. Budgets are either 10^6 , 0, $30n_D$, or $300n_D$.

For the final detailed study, we selected S6, S8, S10, S13, and S14 because they stress spatial imbalance, hub returns, temporal peaks, and relocation-budget sensitivity. We generated 128 instances per selected scenario. The exact IP model from Problem 1 was solved for every generated instance. Because these instances have $n_C = 100$ and $n_K = 100$, the IP is still solvable; its value is therefore the optimal profit and an upper benchmark for any heuristic solution.

We also used a simple feasible lower-bound heuristic. This baseline processes orders chronologically. For each order, it assigns the feasible compatible car requiring the least immediate moving time; if no car is feasible, the order is rejected. It

Scenario	Main factor	Ratio $n_K : n_C$	Levels	Stations	Time/Budget
S1	Baseline	1:1	random	random	random, $B = 10^6$
S2	Ultra-low load	1:10	random	random	random, $B = 10^6$
S3	Low load	1:3	random	random	random, $B = 10^6$
S4	High load	3:1	random	random	random, $B = 10^6$
S5	Level mismatch	1:1	8:2 skew	random	random, $B = 10^6$
S6	Forced relocation	1:1	random	odd/even	random, $B = 300n_D$
S7	One hub	1:1	random	hub 1G	random, $B = 300n_D$
S8	Two hubs	1:1	random	hub 2G	random, $B = 300n_D$
S9	Weekend effect	1:1	random	random	weekend, $B = 10^6$
S10	One peak	1:1	random	random	one peak, $B = 10^6$
S11	Three peaks	1:1	random	random	three peaks, $B = 10^6$
S12	No relocation budget	1:1	random	random	random, $B = 0$
S13	Tight budget	1:1	random	random	random, $B = 30n_D$
S14	Moderate budget	1:1	random	random	random, $B = 300n_D$

has no look-ahead and no repair, so a strong proposed algorithm should clearly beat it. For each instance we report

$$\text{gap} = \frac{\text{IP profit} - \text{algorithm profit}}{|\text{IP profit}|}.$$

Lower is better, and zero means matching the IP optimum.

Scenario	Baseline mean	Baseline std.	Proposed mean	Proposed std.
S6	59.88%	222.65%	0.00%	0.00%
S8	97.53%	550.77%	0.68%	5.36%
S10	27.60%	12.85%	2.16%	2.01%
S13	1063.61%	3967.26%	29.07%	140.57%
S14	60.83%	233.16%	1.10%	4.66%

The results support the algorithm design. In S6, the proposed algorithm matched the IP optimum on all 128 instances. In S8, S10, and S14, its average gap stayed below 2.2%, while the chronological baseline was between 27.6% and 97.5%. S13 is the hardest case because the relocation budget is very tight and the optimal profit is often negative or close to zero; percentage gaps are therefore large and volatile. Even there, the proposed algorithm reduced the average gap from 1063.61% to 29.07%. The experiment therefore shows both sides requested in the project statement: our algorithm is close to the exact optimum on solvable benchmark instances, and it is much better than a simple feasible heuristic. The mean and standard deviation are summarized in the table, the distributional evidence is shown in the histogram figure, and the full set of instances, optimal plans, run-time records, and benchmark rows is preserved in `scenario_study_128_results.tgz` for reproducibility.

Optimal Gap Histograms by Scenario

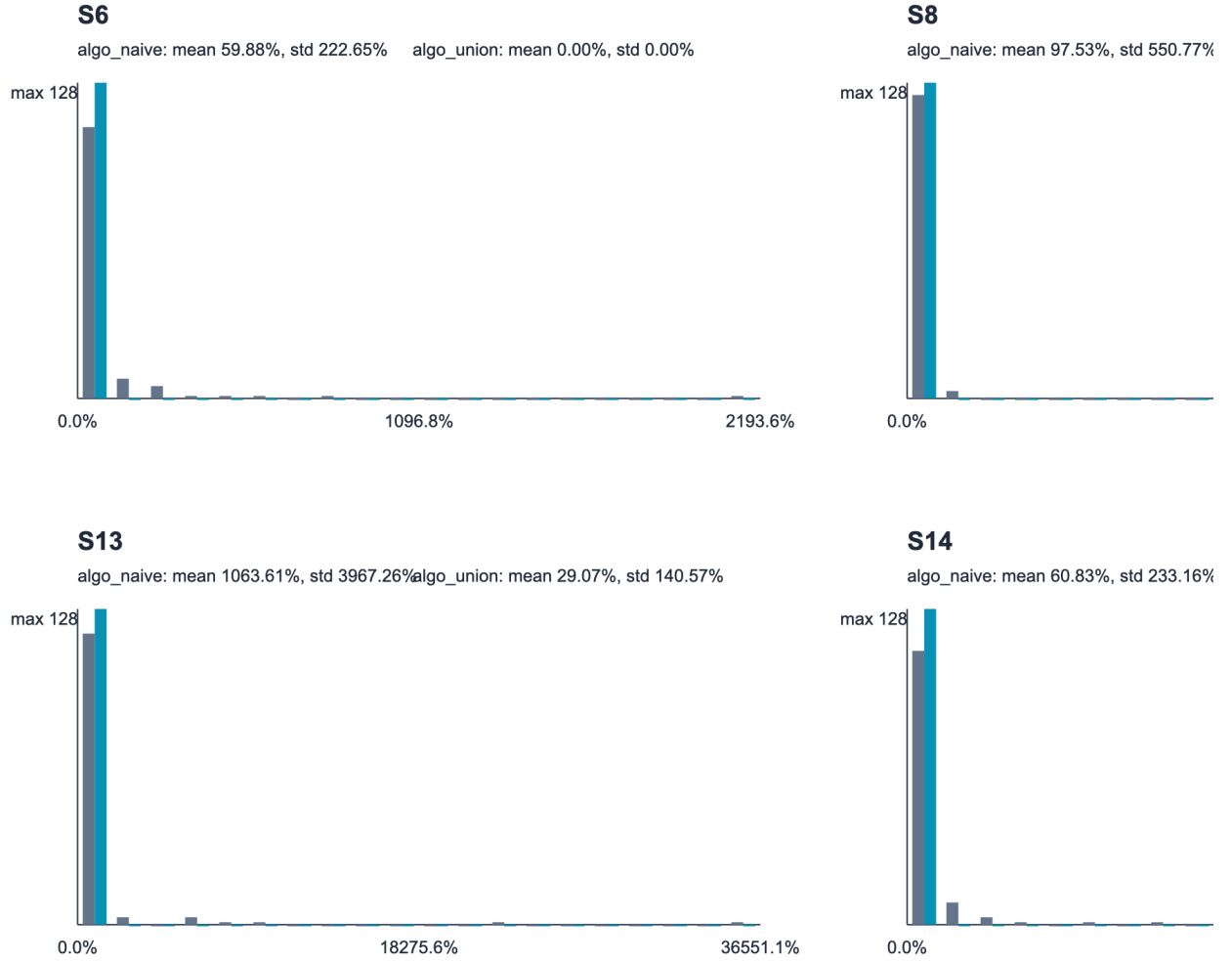


Figure 1: Optimality-gap histograms for the five selected scenarios. Each panel reports the mean and standard deviation of the chronological baseline and our proposed algorithm.